

Concurrency Control for Shared Memory in Multi-Agent Systems

Ayush Gala

agala2@ncsu.edu

North Carolina State University
Raleigh, North Carolina, USA

Ashwattha Phatak

aaphatak@ncsu.edu

North Carolina State University
Raleigh, North Carolina, USA

ABSTRACT

Large language model (LLM) multi-agent systems increasingly rely on shared vector databases for persistent memory. However, traditional distributed lock managers and concurrency control protocols (e.g., 2PL, OCC, MVCC) are structurally blind to “semantic race conditions,” where multiple agents write semantically equivalent payloads expressed in different surface forms. Because these hazards cannot be detected by traditional key-based predicates, we present **DSCC** (Distributed Semantic Conflict Controller), a distributed lock manager that employs cosine similarity over embedding vectors as its conflict predicate. DSCC grants admission only when an incoming embedding lies below a calibrated similarity threshold θ relative to active locks, effectively serializing access based on meaning rather than naming.

DSCC is implemented via a five-node Raft cluster featuring an **ActiveLockTable** that utilizes per-lock FIFO waiter dequeues and a cascading grant rebalancer to ensure fairness while eliminating thundering-herd wakeups. To maintain consistency during consensus, a two-phase admission protocol closes the race window between local admission and durable cluster agreement, preventing “ghost locks” during proposal failures. Evaluation across 13 benchmark scenarios reveals that the choice of embedding model is a critical architectural commitment; while smaller models like *all-minilm* struggle with paraphrase conflicts, *qwen3-embedding:0.6b* achieves a perfect serialization score. Sustaining 32 RPS throughput with a 10 ms median latency, DSCC demonstrates that semantic concurrency control is both feasible and performant for modern LLM-agent workflows.

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; Artificial intelligence; • **Information systems** → **Database concurrency control**; *Distributed database transactions*; • **Theory of computation** → *Distributed algorithms*; *Concurrent algorithms*.

KEYWORDS

distributed concurrency control, semantic locking, multi-agent systems, vector databases, Raft consensus, embedding models, condition variables, FIFO fairness, CAP theorem

1 INTRODUCTION

Large language model agents have moved from one-shot text generators to long-running, collaborating processes that read and write a persistent shared memory [19–21]. In production deployments, this shared memory is almost always a *vector database* [13–16]: each piece of knowledge is stored as a dense embedding vector,

retrieved by approximate nearest-neighbor (ANN) search [17, 18], and read back into a prompt context. As soon as more than one agent shares such a store, a correctness question arises that has no precedent in classical concurrency control: *how does the system prevent two agents from independently persisting the same piece of knowledge expressed in different words?*

A motivating example. Consider an architectural multi-agent system in which a sustainability agent writes the design note “*prioritize passive cooling and low-carbon materials for the civic annex*”, while a parallel agent writes “*evaluate daylight access and sustainable envelope decisions for the annex*.” Both sentences encode the same architectural design intent. In any modern vector store, however, they will be persisted as two distinct points, under two distinct numeric IDs, with two unrelated keys. No row, no key, no znode, no file in the system identifies them as a conflict. A subsequent retrieval will return both, downstream agents will reason over a corpus containing the same intent stated twice, and the resulting plan may double-count it, weight it twice, or—worst case—generate contradictory follow-on writes. This is a *semantic race condition*: two writes whose embedding vectors satisfy $\cos(\mathbf{v}_A, \mathbf{v}_B) \geq \theta$ for some calibrated threshold θ , executed concurrently and admitted by every existing lock manager.

Why classical distributed locking does not apply. Chubby [1], ZooKeeper [2], etcd [3], and Redlock [4] all serialize access to a *lexical name*: a path in a hierarchical namespace, a znode, a key. Their conflict predicate is $\text{NAME}(r_A) = \text{NAME}(r_B)$. Because two paraphrases of the same intent receive different generated keys, they evaluate as non-conflicting and are permitted to proceed in parallel. No tuning, sharding, or hierarchical naming closes this gap; the conflict is a property of the payload’s *meaning*, not its identifier.

Why classical database concurrency control does not apply. Two-phase locking [9], optimistic concurrency control [10], and multi-version concurrency control [11] all reduce conflict to shared physical resources: rows, pages, tuple identifiers, or write-sets expressed over those identifiers. Garcia-Molina and Salem’s semantic concurrency control [12] generalizes this to commutativity over abstract operations, but the operations and their commutativity table are specified by a database designer over structured types, not derived from unstructured natural-language content. None of these mechanisms have a predicate that can be evaluated against the meaning of an English sentence.

Key insight. A continuous embedding space, equipped with cosine similarity, already defines a workable conflict predicate: two payloads are in conflict when their embeddings are within a small

angular distance. The technical problem is not how to compute this—transformer embedders [23, 24, 26] produce such vectors as a byproduct of retrieval anyway—but how to embed it inside a fault-tolerant distributed lock manager whose admission control, fairness, and replication semantics remain sound. *Cosine similarity over embeddings is a usable conflict predicate iff the lock manager itself is engineered around its non-binary, geometric nature.*

Contributions. This paper presents the design, implementation, and empirical evaluation of the **Distributed Semantic Conflict Controller** (DSCC), and makes the following contributions:

- (1) We formalize the *semantic race condition* for shared agent memory and define a cosine-similarity conflict predicate for distributed lock management. We further define *semantic serializability* and the *serialization score*—a recall-style metric that quantifies the fraction of expected conflict pairs that were correctly ordered (§3).
- (2) We present the **ActiveLockTable**, a concurrent in-memory data structure with $O(1)$ keyed lookup, per-lock FIFO waiter dequeues, and per-waiter condition variables. We pair it with the *cascading grant rebalancer*, which scans a freshly orphaned waiter deque front-to-back, migrates waiters to their new strongest blocker when conflicts persist, and grants waiters whose conflict has cleared. Together, these mechanisms eliminate thundering-herd wakeups while preserving FIFO fairness within every conflict group (§6).
- (3) We integrate the lock table with a five-node Raft cluster via a *two-phase admission protocol*: a local pending-slot reservation participates in conflict detection *before* consensus, and a Raft-committed APPLY callback promotes the slot to a real lock. We identify a previously implicit failure mode—the *ghost lock*—in which a failed proposal leaves a stale reservation, and we close it with a deterministic compensating-RELEASE path (§5).
- (4) We conduct an embedding model study across three open-weight models and three threshold values— $3 \times 3 \times 13 = 117$ case runs—and show that *model choice is architectural*: at $\theta=0.75$ on the Paraphrase Gauntlet, all-minilm achieves only a 0.500 serialization score, bge-m3 reaches 0.750, and qwen3-embedding:0.6b is the unique combination that achieves a perfect 1.000. We characterize how θ controls the trade-off between false positives (over-blocking) and false negatives (missed races), and identify a stable operating point (§7).
- (5) We evaluate the full system through 13 curated benchmark scenarios and sustained Locust load, reporting end-to-end latency, fairness, blocked rate, resource utilization, and CAP behavior. Under writes, median latency is 10 ms and P95 is 68 ms—well below the $2 \times \text{HEARTBEAT} = 150$ ms heartbeat-driven upper bound, indicating that most operations clear consensus in a single AppendEntries cycle (§8).

Roadmap. §2 surveys distributed locking, database concurrency control, vector databases, multi-agent memory, Raft, and embedding models. §3 formalizes the semantic conflict model. §4 describes the system architecture and its four runtime layers. §5 dissects the Raft consensus layer and the two-phase admission protocol. §6 presents the ActiveLockTable in full, including the rebalancer and

a correctness argument. §7 treats the embedding model as a first-class architectural component. §8 reports the empirical evaluation. §9 discusses limitations and future work, and §10 concludes.

2 RELATED WORK

DSCC sits at the intersection of four research traditions: distributed locking, database concurrency control, vector retrieval, and agent memory architectures. We review each and highlight the conceptual gap that DSCC is designed to fill, then summarize the Raft consensus protocol and the role of embedding models in defining the geometry of a lock space.

2.1 Distributed Locking Systems

Chubby [1] was the canonical distributed lock service: clients acquire coarse-grained advisory locks on named nodes in a tiny filesystem-like namespace, with a Paxos [5] replicated state machine guaranteeing that at most one client holds a lock at a time. The conflict predicate is identity equality on the lock path. **ZooKeeper** [2] generalized this to wait-free hierarchical coordination using ephemeral, sequenced *znodes*; distributed mutexes, leader election, and barriers are built by applications on top, but the underlying conflict predicate remains lexical equality on a *znode* path. **etcd** [3] replaces this with a Raft-replicated key-value store and exposes leases and compare-and-swap on string keys. **Redlock** [4] relaxes consistency for availability: a lock is held when a majority of independent Redis instances have stored the same string key under TTL.

These systems are mature and operationally invaluable, but their model of a conflict is structurally lexical: $\text{KEY}(r_A) = \text{KEY}(r_B)$. They cannot serialize two writes whose payloads encode the same intent under different keys, because they never look at the payload. DSCC retains the quorum-replicated lock-table architecture of these systems but replaces the predicate with cosine similarity over embedding vectors.

2.2 Database Concurrency Control

In relational databases, concurrency control mechanisms ensure serializability by reasoning about reads and writes over identified data items. **Two-phase locking** [9] acquires shared or exclusive locks on rows, pages, or tables, in a strictly two-phase acquire-then-release order. **Optimistic concurrency control** [10] validates a transaction's read- and write-sets at commit time, aborting any transaction whose read-set intersects a committed write-set. **Multi-version concurrency control** [11] keeps per-transaction snapshots indexed by primary key, so reads do not block writes. In every case, the conflict predicate reduces to set intersection over identifiers. **Semantic concurrency control** [12] introduced the idea that two operations may run concurrently if their effects commute, but commutativity is specified declaratively over a designer-supplied operation algebra (e.g., increment, decrement, push, pop) and is undefined for free-form natural-language content.

DSCC operates in a regime where the conflicting items do not have a shared identifier and may not even share a vocabulary: two paraphrases of the same sentence are physically distinct points in a vector store. We therefore need a predicate that operates on the geometric *representation* of the payload rather than on its identity, and we need it to be evaluable in real time on the admission path.

2.3 Vector Databases and ANN Search

Vector databases such as **Qdrant** [13], **Weaviate** [14], **Pinecone** [15], and the **pgvector** [16] extension to PostgreSQL store dense embedding vectors alongside scalar payloads and support fast approximate nearest-neighbor retrieval. Two indexing families dominate: graph-based **HNSW** [18], which builds a hierarchical navigable small-world graph and answers k -NN queries in expected logarithmic time, and quantization-based **FAISS** [17] indexes (IVF, IVF-PQ, HNSW-PQ), which trade exact distance for memory by partitioning the space and quantizing vectors. Both families optimize *retrieval*, not *admission*: their concurrency abstractions stop at the collection level, with point-level upserts treated as independent. None of them detect that two concurrent upserts encode the same intent. DSCC complements the vector database rather than replacing it: Qdrant remains the durable store, while DSCC sits in front of every write and decides whether admission is safe.

2.4 Multi-Agent Memory Architectures

Modern agent frameworks delegate memory consistency entirely to the underlying store. **LangGraph** [19] models agent flows as graphs of nodes with shared state; concurrent state updates are reconciled either by last-writer-wins or by user-supplied reducers, neither of which can detect semantic duplicates. **AutoGen** [20] composes LLM agents through conversation patterns and provides no memory-write arbitration. **Generative Agents** [21] demonstrated that long-running simulated agents benefit from a shared “observation memory” but ran a single observation pipeline, sidestepping the multi-writer problem. **ReAct** [22] and similar single-agent reasoning architectures assume a single thread of control. As soon as agents are scaled out across multiple processes or hosts, all of these frameworks are exposed to the semantic race condition we address.

2.5 Raft Consensus

Raft [6] is a consensus protocol designed for understandability, implementing the same safety guarantees as Paxos [5] through a strong leader and an explicit log abstraction. Cluster nodes are in one of three states—FOLLOWER, CANDIDATE, or LEADER. Time is divided into terms, each with at most one leader. **Leader election** is triggered by an election timeout; a candidate increments the current term, votes for itself, and solicits votes from peers. A vote is granted iff the candidate’s log is at least as up-to-date as the voter’s (by last-log-term, then last-log-index) and the voter has not voted in this term. **Log replication** proceeds via AppendEntries RPCs from the leader, each carrying the term, the index and term of the immediately prior entry, and a batch of new entries. A follower accepts a batch iff its local log matches at the prior index and term, otherwise it rejects with a hint index that lets the leader back up efficiently. **Commitment** is defined by quorum: an entry is committed when it has been replicated to a majority *and* the leader has at least one entry from its current term in the log—this prevents the well-known “Raft figure 8” anomaly in which a stale entry is incorrectly counted as durable.

DSCC implements its own Raft layer (rather than using etcd or hashicorp/raft) to obtain tight integration with the lock table’s APPLY callback. We inherit Raft’s safety guarantees—at most one

leader per term, log matching, leader completeness—and add to it (i) a deterministic APPLY callback into ActiveLockTable that promotes pending slots into real locks, and (ii) the orchestration logic surrounding the two-phase admission protocol. We deliberately accept several known engineering simplifications (sequential vote collection, no pre-vote) and discuss their implications in §5.6 and §9.

2.6 Embedding Models for Semantic Similarity

A modern embedding model is a transformer encoder fine-tuned with a contrastive objective so that semantically equivalent sentences map to vectors with high cosine similarity. Sentence-BERT [23] established the recipe; subsequent models such as **BGE** [24], **E5** [25], and the **Qwen3** [26] embedding family scale capacity and multilingual coverage. Cosine similarity in this space approximates *semantic equivalence*: paraphrases cluster, synonyms cluster, and unrelated sentences spread out. The geometry of the resulting space, however, is a property of the model. A small, distilled encoder collapses paraphrases into vectors that are similar enough for human inspection but not similar enough to clear a high cosine threshold. A larger encoder preserves finer angular distinctions and produces tighter paraphrase clusters but is slower to evaluate.

For DSCC, this means the embedding model is not an interchangeable component: it *is* the conflict predicate. §7 quantifies how dramatically model choice alters the semantics of the lock space, and identifies the only model in our study that yields a zero-violation operating point.

Summary. The four research traditions surveyed above each contribute one piece of the puzzle—fault-tolerant replicated locking from distributed systems, principled admission control from databases, dense retrieval geometry from vector stores, and consensus from Raft—but none combine them in a way that detects semantic conflicts on the admission path. The remainder of the paper presents that combination.

3 PROBLEM FORMULATION

We formalize the system model, the semantic conflict predicate, the resulting race condition, and a notion of semantic serializability that we use as the primary correctness criterion in evaluation.

3.1 System Model

We assume a set of n agents $\mathcal{A} = \{a_1, \dots, a_n\}$, each running as an independent process and writing through DSCC to a shared *vector database* $\mathcal{D}$. Agents communicate with DSCC through gRPC and otherwise share no state. Each write request r carries a natural-language payload $s_r \in \Sigma^*$ over an unspecified alphabet.

A deterministic *embedding function* $\text{EMBED} : \Sigma^* \rightarrow \mathbb{R}^d$ maps a payload to a fixed-dimension unit-normalized vector $\mathbf{v}_r = \text{EMBED}(s_r)$. We treat EMBED as a black box supplied by the deployment (see §7); it is the same function for every agent in a deployment, and we assume that a small numerical drift due to floating-point non-determinism is below the threshold sensitivity bands we calibrate in §7.

Time is partitioned into a logical event sequence per agent. An agent issues each request to DSCC's leader-aware proxy, which forwards it to the current Raft leader; the leader returns an `AcquireResponse` that records, among other timestamps, $\text{LOCK}_{\text{acq}}(r)$ and $\text{LOCK}_{\text{rel}}(r)$ delimiting the lock-hold interval. The agent's persistence operation against $\mathcal{D}$ is bracketed inside this interval. **We assume crash-stop failures of nodes (no Byzantine behavior) and a partially synchronous network with bounded but unknown delay; this is the standard Raft assumption.**

3.2 Semantic Conflict

Two requests r_A and r_B with embeddings $\mathbf{v}_A, \mathbf{v}_B \in \mathbb{R}^d$ are *semantically conflicting at threshold θ* iff

$$\cos(\mathbf{v}_A, \mathbf{v}_B) = \frac{\mathbf{v}_A \cdot \mathbf{v}_B}{\|\mathbf{v}_A\| \|\mathbf{v}_B\|} \geq \theta, \quad (1)$$

where $\theta \in (0, 1]$ is a deployment-calibrated parameter. The numerator is the standard Euclidean dot product and the denominator the product of ℓ_2 norms; cosine therefore measures the angle between the two vectors independent of magnitude. Geometrically, θ defines a *spherical cap* centered at $\mathbf{v}_A / \|\mathbf{v}_A\|$ on the unit sphere S^{d-1} ; any point inside the cap is in conflict. Semantically, θ controls how aggressively paraphrases are detected: high θ (e.g. 0.95) requires near-identical phrasing; low θ (e.g. 0.55) flags broadly related ideas.

The threshold is *not* an absolute semantic boundary that holds across deployments. It is a parameter calibrated against the geometry produced by the chosen embedding model on the chosen workload. In our evaluation (§7), we sweep $\theta \in \{0.55, 0.75, 0.95\}$ and show that the same nominal value yields very different operating points across models.

Threshold asymmetry. The conflict check in DSCC always uses the *incoming* request's θ , not the active lock's θ . A conservative request ($\theta_{\text{in}} = 0.55$) may therefore be blocked by an active lock that itself was acquired with $\theta_{\text{lock}} = 0.95$ even though the active lock would not have considered the new request a conflict. We adopt this asymmetric semantics deliberately so that a single conservative client can protect itself without requiring all peers to lower their thresholds; the trade-off is discussed in §9.

3.3 Semantic Race Condition

A *semantic race condition* occurs when two semantically conflicting requests r_A, r_B have overlapping lock-hold intervals $[\text{LOCK}_{\text{acq}}(r_A), \text{LOCK}_{\text{rel}}(r_A)] \cap [\text{LOCK}_{\text{acq}}(r_B), \text{LOCK}_{\text{rel}}(r_B)] \neq \emptyset$. Concretely, both writes commit to $\mathcal{D}$ without an enforced ordering; $\mathcal{D}$ ends up with two duplicate-intent points, downstream readers observe both, and any reasoning over the corpus is corrupted by the duplication. The duplication is permanent because vector stores have no notion that the two points are conflicts: a deduplication pass would itself need to re-embed and threshold-compare, which is precisely the predicate DSCC enforces on admission.

Concrete example. Agent A submits the payload “review the massing concept and prioritize passive cooling, daylight access, and

low-carbon material options for the civic annex”; Agent B, simultaneously, submits “re-examine the early massing study with an emphasis on sustainable envelope decisions and reduced embodied carbon for the annex program.” Both encode the same architectural design intent. A high-quality embedder maps them to vectors with $\cos \approx 0.83$. At $\theta=0.75$, Equation 1 flags this pair as a conflict. Without DSCC, both requests reach Qdrant with distinct numeric IDs; the two points co-exist forever. With DSCC, the second request is queued behind the first on a per-lock waiter deque (§6) and only proceeds after the first releases. Crucially, no key-based mechanism—neither row IDs in PostgreSQL, nor znode paths in ZooKeeper, nor IDs in Qdrant—can detect this pair, because no key is shared.

3.4 Semantic Serializability and the Serialization Score

A request schedule is *semantically serializable at θ* iff, for every pair of requests that would conflict at θ , one's lock-hold interval ends strictly before the other's begins. Equivalently, the conflict graph induced by Equation 1 is correctly serialized.

We measure how close a run comes to this ideal with the *serialization score*:

$$\text{ss}(\theta) = 1 - \frac{\text{VIOLATIONS}(\theta)}{\text{CONFLICT_PAIRS}(\theta)}, \quad (2)$$

where $\text{CONFLICT_PAIRS}(\theta) = |\{(r_A, r_B) : \cos(\mathbf{v}_A, \mathbf{v}_B) \geq \theta\}|$ is the count of pairs the predicate flags as conflicting, and $\text{VIOLATIONS}(\theta)$ is the count of those pairs whose lock-hold intervals nevertheless overlap. $\text{ss} = 1.000$ means zero semantic race conditions were observed in the run; $\text{ss} = 0$ means the system effectively does not serialize at all. We treat ss as a recall metric: any violation is a missed safety case.

A complementary metric is the *distinct parallelism rate*, which counts what fraction of the pairs that should *not* conflict actually ran in parallel. Maximizing parallelism on distinct pairs while pinning $\text{ss} = 1.000$ is the central correctness-throughput trade-off explored in §7.

3.5 Design Goals

DSCC is designed against four explicit goals:

- (1) **Correctness**—semantic serializability: $\text{ss}(\theta_{\text{calibrated}}) = 1.000$ on the Paraphrase Gauntlet.
- (2) **Parallelism**—distinct, non-conflicting writes execute concurrently: distinct parallelism rate $\rightarrow 1$ on the Cross-Domain Flood at the calibrated θ .
- (3) **Fairness**—FIFO ordering within every conflict group: admission of two waiters w_i, w_j on the same lock with i enqueued before j implies w_i acquires before w_j , even across rebalances; we verify this through queue-position and queue-hop trace fields exposed in each `AcquireResponse`.
- (4) **Fault tolerance**—a five-node Raft cluster with $\text{QUORUM} = 3$ tolerates the simultaneous failure of any two nodes without loss of admitted-lock state; this is the standard safety property we inherit directly from Raft.

These goals frame the architecture, the lock-table design, and the evaluation that follow.

4 SYSTEM ARCHITECTURE

DSCC is built as four cleanly separated runtime layers, all communicating over gRPC: (1) the *embedding service*, (2) the *leader-aware proxy*, (3) the *five-node DSCC cluster*, and (4) the *Qdrant vector database* (Figure ??). The separation is functional, not just deployment-level: each layer has a well-defined responsibility, a narrow gRPC contract, and an independent failure mode. We discuss each in turn after walking through the full data flow for a single write.

End-to-end data flow for a single write.

- (1) The agent reads its payload (e.g., from a JSON template in `demo_inputs/`) and issues an HTTP POST to the embedding service with the chosen model name. The service returns a d -dimensional vector.
- (2) The agent constructs a gRPC `AcquireRequest` containing `agent_id`, the embedding, the raw payload text (for telemetry), the operation type (`WRITE` or `READ`), and a client timestamp. It sends the RPC to `dsc-proxy` on port 50050.
- (3) The proxy looks up its cached leader address and forwards the request via `ForwardWithLeaderRetry`. On `NOT_LEADER`, the proxy refreshes its leader cache and retries up to three times within the request deadline.
- (4) The leader's `LockServiceImpl::AcquireGuard` handler runs the three-phase protocol (§5.2): admission against the `ActiveLockTable`, Raft `ACQUIRE` consensus, and `APPLY`-callback promotion of the pending slot. It then performs the actual Qdrant `PUT /collections/{col}/points?wait=true` (or, for reads, `POST .../points/search`).
- (5) After the Qdrant write, the leader proposes a Raft `RELEASE` entry; on commit, every node's `APPLY` callback removes the lock from its local table, and the leader's `rebalance_waiters` locked hands granted waiters their per-CV notifications. The agent's gRPC response carries every timestamp in the pipeline plus a trace block (`wait_position`, `queue_hops`, `wake_count`, `blocking_similarity`, `blocking_agent_id`, `active_lock_count`) for offline analysis.

Failure handling at the architectural level. A leader change interrupts in-flight requests at the gRPC layer; the proxy observes `UNAVAILABLE` or `NOT_LEADER`, refreshes the leader cache via `GetLeader`, and retries. A network partition that strands the leader in a minority causes its proposals to never reach quorum; `Propose` times out, the handler appends a compensating `RELEASE` via `AppendLocalEntry` (§5.4), and returns `UNAVAILABLE`. A node failure inside a healthy majority is absorbed by Raft replication: as long as `QUORUM = 3` acknowledges each entry, progress continues. On rejoin, the failed node catches up via backward-walking `AppendEntries` with the fast conflict-index hint.

Deployment topology. The full stack is described by a single `docker-compose.yml`: one Qdrant container (port 6333), one Ollama container (`host:7997 → container:11434`), five `dsc-node` containers (each exposing `50051+i` for the client gRPC service and `50061+i` for the inter-node Raft service), and one `dsc-proxy` container (port 50050). Containers are network-isolated and each runs

the same image, with behavior parameterized through environment variables (`theta`, `lock-hold duration`, `Raft timeouts`). This topology was chosen to enforce that *node-to-node communication uses the same gRPC stack as client-to-leader communication*: there is no shortcut for replication, and no in-process binding lets Raft messages bypass the on-the-wire path. The identical deployment is used for the `e2e` benchmark, the `Locust` load tests, and the live demo, so all measured numbers reflect realistic gRPC latency.

4.1 Embedding Service (Ollama)

The embedding service is an Ollama instance that exposes the OpenAI-compatible `/v1/embeddings` endpoint. A single container hosts all three models (`all-minilm`, `bge-m3`, `qwen3-embedding:0.6b`); agents and benchmark drivers select a model per request via the `model` field. Model weights are cached in a Docker volume so cold starts only happen on first run.

Why a separate service? The architectural goal is that *the lock table never interprets natural language*. The `AcquireRequest` carries a vector and a scalar threshold; the lock table compares floats. This clean separation has three benefits. First, the embedding model can be replaced at deployment time without recompiling the lock manager—the embedding study in §7 is conducted entirely by changing the model name passed in each request. Second, embedding latency (10–133 ms p50) is incurred on the agent side, before the gRPC call, so the leader's hot path is not lengthened by tokenization or inference. Third, swapping the embedder for a future modality (e.g., a multimodal encoder for image payloads) is a configuration change, not a redesign.

4.2 Leader-Aware Proxy

The proxy (`src/proxy_main.cpp`, `src/proxy_service_impl.cpp`) is a stateless C++ gRPC server that sits in front of the cluster on port 50050. Three concerns motivate its existence: (i) clients should not have to discover the current Raft leader themselves; (ii) leader changes should not surface as application-level errors; (iii) gRPC channels to backend nodes should be reused across requests to avoid TCP/TLS handshake costs.

Leader poll loop and caching. A dedicated thread runs `LeaderPollLoop`, which calls `RaftService::GetLeader` on every backend every `LEADER_POLL_MS=100` ms. `GetLeader` returns either “I am leader” or “the current leader is at `addr`”; the first response of either kind that arrives within `LEADER_RPC_TIMEOUT_MS=750` ms wins, and the cached leader address is updated atomically. The poll interval and the heartbeat interval (75 ms) are deliberately mismatched so that the proxy's view converges to the cluster's leader within a few election timeouts even under heartbeat-rate churn.

Forwarding and retry. `ForwardWithLeaderRetry<T>(req, rpc)` is a templated helper that loops up to three times: it (a) ensures a cached leader exists, refreshing if stale; (b) creates a `LockService::Stub` on the cached channel; (c) calls `rpc` with a per-attempt deadline of `REQUEST_TIMEOUT_MS=35` s. On success it returns. On `FAILED_PRECONDITION` carrying “`NOT_LEADER`”, the proxy reads the leader-address trailing-metadata key, replaces its cached leader with that hint, and retries. On `UNAVAILABLE` or `DEADLINE_EXCEEDED`, it forces a leader

refresh and retries. After three failures, the original status is propagated to the client.

Lazy channel creation. gRPC channels are stored in an `unordered_map<string, shared_ptr<Channel>>`. A channel to a backend is created on first use and never closed; subsequent requests to the same backend reuse it. This means a leader flip only requires the proxy to switch which channel it dials—there is no reconnection cost, just a different stub on an existing channel.

Why a proxy at all? Without the proxy, every client SDK would need to embed leader-discovery logic, plus retry-on-redirect, plus channel caching. By centralizing these concerns in a single C++ binary, we keep clients simple (single endpoint, no Raft awareness) and we make the discovery latency a property of the cluster rather than of each client. The proxy is itself stateless aside from its in-memory leader cache, so it can be replicated horizontally behind a TCP load balancer without coordination.

4.3 The DSCC Cluster

Each of the five dsccl-node processes runs an identical binary that wires together a `RaftNode`, an `ActiveLockTable`, a `LockServiceImpl` (the gRPC handler for client requests), and a `RaftServiceImpl` (the gRPC handler for inter-node Raft RPCs). The node listens on two ports: 50051 for clients and 50061 for peers.

Leader-only operations. `AcquireGuard` and `ReleaseGuard` are leader-only. A non-leader handler immediately returns `FAILED_PRECONDITION` with the leader address in trailing metadata. The proxy interprets this as a redirect. We chose strict leader-only admission rather than follower forwarding because admission requires running `wait_for_admission`, which mutates the lock table; running this on followers would either require synchronous round-trips on the admission path (defeating the purpose of replication) or would create races with Raft `APPLY` ordering. Read-only operations would be a candidate for follower service, but the current implementation routes them through the leader as well, since reads also pass through the semantic admission predicate (§9).

State replication and consistency. The lock table on each node is rebuilt from the same Raft log via the `on_commit_callback`: `ACQUIRE` entries trigger `apply_acquire`, `RELEASE` entries trigger `apply_release`. Because Raft guarantees that all nodes apply the log in the same total order, every node's lock table converges to the same state at the same commit index. The leader's table additionally contains *pending* slots for in-flight admissions; followers see only committed locks.

Forward pointer. §5 dissects the Raft consensus layer and shows how the two-phase admission protocol, the pending-slot mechanism, and the ghost-lock prevention path together close the race window between local admission and durable cluster agreement. §6 then dissects the `ActiveLockTable` in detail.

5 RAFT CONSENSUS LAYER

Raft replicates the lock-table mutations across the five-node cluster and produces the strict total order that the `APPLY` callback requires. We describe the cluster configuration and threading model,

the two-phase admission protocol, the pending-slot mechanism that closes the admission/consensus race, the ghost-lock prevention path, the release protocol, and the engineering simplifications we knowingly accepted.

5.1 Cluster Configuration and Threading Model

The cluster has $N=5$ nodes, $QUORUM = 3$, and tolerates two simultaneous failures. Each node runs three background threads on top of the gRPC server thread pool:

- `HearbeatLoop` (75 ms period). On the leader, it spawns one thread per peer per round and issues `AppendEntries` (with or without entries) to keep the followers' election timers reset and to drive replication.
- `ElectionTimerLoop` (10 ms poll, 600–1000 ms randomized deadline). On non-leaders, if no `AppendEntries` has arrived before the deadline, the loop transitions the node to `CANDIDATE` and starts an election.
- `ApplyLoop`. Waits on `apply_cv_for last_applied_ < commit_index_`; on each iteration, pops the next entry, calls the registered `on_commit_callback`, and signals back.

Rationale for the timing parameters. The 75 ms heartbeat sets the floor on commit latency: a write incurs at least one heartbeat round-trip on the critical path, plus replication RPCs. A faster heartbeat reduces commit latency but increases idle CPU and network usage; 75 ms is a standard mid-point used in production Raft deployments such as etcd and works well for our local-network testbed. The 600–1000 ms randomized election timeout is approximately $8\times$ the heartbeat—enough that a single dropped heartbeat does not trigger an election, but small enough that a leader failure is detected within a second. The 150 ms per-RPC timeout is short enough that a hung peer does not wedge a heartbeat round, and long enough that a healthy local network never spuriously times out.

5.2 Two-Phase Admission Protocol

A successful `AcquireGuard` call traverses three phases: an `ADMISSION` phase that runs locally against the lock table, a `CONSENSUS` phase that replicates the decision through Raft, and a `PROMOTION` phase that fires on every node via the `APPLY` callback.

Phase 1 — Admission. The handler validates the request and calls `lock_table_>wait_for_admission(agent_id, embedding, theta)` (Algorithm ?? line 6). This call blocks the calling gRPC worker thread inside a per-waiter condition variable until no active or pending lock conflicts with the incoming embedding. On clearance, it *atomically* inserts a *pending slot*: a `SemanticLock` entry with `pending=true`. The slot participates in subsequent conflict scans (§5.3) but is not yet replicated.

Phase 2 — Consensus. The handler builds a `LogEntry{op=ACQUIRE, agent_id, embedding, theta, term}` and calls `raft_>Propose(entry, timeout, &idx)`. `Propose` appends the entry to the local log, then enters a parallel-replication loop: one thread per peer issues `AppendEntries` with the new entry; after each RPC, the leader advances `commit_index_` if a quorum of `match_index_` values has caught up *and* the entry's term matches `current_term_`. The handler then calls `raft_>WaitUntilApplied(timeout)`, which blocks on `apply_cv_` until `last_applied_` reaches

idx. PROPOSE or WAITUNTILAPPLIED returning false indicates a failed consensus—either the leader stepped down, the network split, or a peer is too far behind. The handler then takes the ghost-cleanup branch (lines 9–11).

Phase 3 — Promotion. On commit, every node's ApplyLoop eventually pops the entry and fires on_commit_ (registered in main.cpp). On the leader, apply_acquire finds the existing pending entry by agent_id, sets pending=false, and updates the centroid/threshold from the committed values; the waiter deque on the slot is unchanged. On a follower, no pending slot exists, so apply_acquire inserts a fresh SemanticLock with pending=false. The leader's handler now proceeds to Qdrant.

Walkthrough. A single AcquireGuard for an unconflicted write incurs (i) one local CV check (sub-microsecond if the table is empty), (ii) one parallel AppendEntries round to four peers, on which the slowest single round dominates (typically ≤ 75 ms), (iii) the local Qdrant PUT (5–10 ms typical), and (iv) the RELEASE consensus round. Steps (ii) and (iv) are the irreducible floors on write latency, and together they bound the observed P95 of 68 ms in our Locust write workload (§8).

5.3 Pending Lock Semantics

The pending flag exists to close a specific admission/consensus race. Suppose Phase 1 succeeds and the handler calls Propose. While the proposal is being replicated, the leader's gRPC server is still serving other agents. A second agent B with a payload that conflicts with A's arrives. If A had not inserted any reservation, B's call to find_conflict_locked would scan only the committed locks and return “no conflict”—B would then enter Phase 2 in parallel with A. Both proposals could commit before either applies, and the lock table would contain both A and B as concurrent winners of the same semantic region.

The pending slot, inserted under μ_* at the end of Phase 1, makes this impossible. The slot has the same shape as a real lock—an agent_id, a centroid, a threshold, and a waiter deque—differing only in pending=true. Because find_conflict_locked does not distinguish pending from real, B will see A's pending slot, fail the conflict check, and either enqueue behind A or be requeued onto a stronger blocker on its next rebalance. The window between admission and durable agreement is reduced to a single mutex hold inside wait_for_admission.

The interaction with the conflict predicate is subtle but consistent: the predicate is evaluated against the centroid stored on the slot, which is the embedding the requester supplied; once the slot is promoted, the centroid does not change. Pending slots therefore have the same geometric reach as fully committed locks, and any waiter that conflicts with a pending slot will continue to conflict after promotion.

5.4 Ghost Lock Prevention

A *ghost lock* is a pending slot that was never promoted but also never removed. It can arise in two ways: (i) Propose times out or fails because consensus could not be reached; (ii) the leader steps down (loses leadership) between Phase 1 and the APPLY callback. If the slot is left in place, every future request that conflicts with it

will block forever; the system effectively wedges on that semantic region.

DSCC handles both cases through a deterministic compensating-RELEASE path:

- (1) On Propose or WaitUntilApplied failure, the handler calls remove_pending(agent_id). remove_pending is a no-op if the entry has already been promoted (pending=false), and otherwise it (a) erases the entry from active_, (b) detaches its waiter deque, and (c) reuses rebalance_waiters_locked to redistribute or grant those waiters.
- (2) Independently, the handler appends a compensating RELEASE entry locally via AppendLocalEntry, which adds the entry to the local Raft log without waiting for quorum. If the original ACQUIRE did happen to have replicated to a subset of followers before failing, the compensating RELEASE will follow it through the normal heartbeat replication path and remove the lock on those followers when it commits. If the ACQUIRE never replicated, the compensating RELEASE applies harmlessly via the “release for unknown agent_id” branch in apply_release (which logs a warning and returns).

A second cleanup mechanism exists for case (ii): when a node becomes leader, it calls begin_leader_sweep() on its ActiveLockTable, which (a) gates new wait_for_admission calls behind a condition variable and (b) returns a snapshot of all currently held agent_ids. The new leader then proposes a RELEASE for each, restoring the cluster's lock table to a consistent baseline. end_leader_sweep() then ungates admissions. The combination of the per-request compensating path and the per-leader sweep covers all ghost-lock scenarios we have identified.

5.5 ReleaseGuard Protocol

On the success path, AcquireGuard ends with a call to commit_release_once(), which is also bound to a ScopeExit guard so it fires on every error path. commit_release_once proposes a LogEntry{op=RELEASE, agent_id} via Raft, waits until applied, and returns. apply_release on every node delegates to ActiveLockTable::release, which performs three steps under μ_* : (a) remove_lock_locked erases the SemanticLock entry and detaches its waiter deque, (b) rebalance_waiters_locked processes the orphaned deque (§6.4), and (c) on the leader only, the granted-waiter list is held until μ_* drops, then each granted waiter's CV is notified. The CV-after-mutex ordering is critical for performance and correctness, as discussed in §6.6.

The explicit ReleaseGuard RPC exists primarily for testing and agent-side error recovery; in normal operation, the RELEASE is fired automatically by AcquireGuard after the Qdrant write completes (or fails). This keeps the lock-hold window tightly bounded to the actual write duration plus the optional LOCK_HOLD_MS artificial delay used for benchmark scenarios.

5.6 Known Limitations of the Raft Implementation

We deliberately accepted four engineering simplifications and document each here.

Sequential vote collection. StartElection sends RequestVote RPCs serially to all peers, giving worst-case election time $O(N \times \text{rpc_timeout})$. On a healthy local network all RPCs return in under 5 ms each and the parallel-versus-serial gap is invisible; on a high-latency WAN this would significantly delay recovery. The fix is straightforward: spawn one thread per peer or use async/coroutines and collect responses on a shared atomic counter. We did not implement this because the production-grade fix also requires a *cancellation* story for outstanding RPCs once quorum is reached, and that interacts non-trivially with the configured `rpc_timeout` budget.

Absence of pre-vote. A node that was partitioned and incremented its term in isolation will, on rejoin, immediately step the current healthy leader down by sending RequestVote with the larger term. This causes an unnecessary election. The pre-vote extension to Raft [7] runs a hypothetical “would-you-vote-for-me” round before incrementing the term; only if a quorum says yes is the term incremented. Production Raft implementations in etcd, CockroachDB [8], and Consul implement pre-vote for exactly this reason. We mark it as future work.

gRPC backoff sensitivity. A single dropped packet can push a gRPC channel into its exponential backoff window, silencing heartbeats to that peer. If this happens on the leader’s channel to enough peers, its election timer can fire and trigger a spurious election. Channel-level keepalive tuning (`GRPC_ARG_KEEPALIVE_TIME_MS`) and an application-level retry budget mitigate this; we recommend these settings for production use but currently rely on default channel parameters.

WAL without snapshots. The Raft log is `std::vector<LogEntry>` in process memory, with a sentinel at index 0. There is no WAL and no snapshot mechanism: on process restart, a node loses its log and rejoins as a fresh follower, requiring a full replay from the leader. `InstallSnapshot` is wired for the term check but does not actually install snapshot data. A production fix requires a durable WAL (e.g., a memory-mapped file with `fdatasync` on commit), a periodic snapshot of the lock table, and a working `InstallSnapshot` implementation that the leader can use when a follower’s log has been compacted away.

Forward pointer. The Raft layer guarantees that every node sees the same totally ordered log of `ACQUIRE/RELEASE` entries; the lock table on each node is therefore a deterministic function of that log. §6 now dissects how the lock table itself is structured to make admission, fairness, and rebalancing efficient.

6 THE ACTIVELOCKTABLE

The `ActiveLockTable` is the central technical contribution of this paper. It is the in-memory data structure that answers, for every incoming request, two questions: “Does this request conflict with any active or pending lock?” and, if so, “Which lock should this request wait behind, and in what queue position?” It also implements the cascading grant rebalancer, which preserves FIFO fairness within each conflict group while granting as many waiters as possible per release.

6.1 Design Motivation: The Thundering-Herd Failure of the Single-CV Design

The earliest implementation used the simplest possible representation: a flat `std::vector<SemanticLock>` guarded by a single mutex `mu_` and a single global condition variable `cv_`. On every release, the code called `cv_.notify_all()`; every sleeping waiter then woke up, re-acquired `mu_`, and re-ran the conflict scan. This was correct, but it had three pathological behaviors.

Thundering herd. Consider Case 1 (*The Thundering Herd*): 13 agents arrive simultaneously with the same payload, all conflict, and one wins admission while the other 12 enqueue. When the winner releases, `notify_all` wakes all 12. They contend on `mu_` in serial; each acquires it, runs the $O(n)$ scan, finds it still conflicts with the next granted lock, and goes back to sleep. Eleven of the twelve wakeups produced no progress, and they together held `mu_` for roughly $11 \times O(n)$ time, blocking the granted waiter from making forward progress. Throughput degrades nearly linearly with queue depth.

Quantitative characterization. In a microbenchmark with 13 agents on the same conflict region, the single-CV design produced a wakeup-to-grant ratio of approximately 12:1: 12 wasted wakes per 1 useful grant. Each wasted wake added $\approx 0.3\text{--}1.5$ ms of mutex hold time (depending on n), so the effective serialization rate was bounded around 100–200 grants/second even on an otherwise idle machine. Under our per-waiter CV design, the ratio is 0:1: only the next-to-grant waiter is woken, and it proceeds directly to Phase 2 without re-running the conflict scan.

FIFO violation. With `notify_all`, the order in which threads regain `mu_` is OS-scheduler-dependent. A thread that was 12th in the queue might re-acquire `mu_` before a thread that was 2nd, run the conflict scan, and find a transient gap (say, a third lock just released). It would then claim admission ahead of the 2nd-in-line waiter, violating FIFO fairness. This is observable as agents reporting `wait_position=12` but `queue_hops=0` and a low `lock_wait_ms`—a fast, unfair grant.

These three behaviors together motivate the per-lock waiter deque and per-waiter CV design that follows.

6.2 Data Model

Field-by-field invariants. `WaitQueueEntry::waiting_agent_id` identifies the request and is used as the key when the waiter is later promoted to a lock. `embedding` and `theta` are immutable for the lifetime of the waiter and capture the conflict predicate input the request was admitted under. `cv` is private to the waiter; only the rebalancer ever calls `notify_all` on it. `ready` gates the wait predicate so that spurious wakeups (none are expected, but the code is defensive) re-loop into the conflict scan. `granted` distinguishes the *ownership-handoff* wakeup (the rebalancer has already inserted a pending slot for this waiter, so it must skip the scan and proceed directly to Phase 2) from a *rescan* wakeup (the waiter must rerun `find_conflict_locked`). `queue_hops` is incremented every time the rebalancer moves the waiter from one lock’s deque to another, providing per-waiter telemetry on rebalancing churn.

`SemanticLock::agent_id` is the key in the `unordered_map`, giving $O(1)$ insertion, removal, and lookup. `centroid` is the lock's representative vector; it is set to the admitting request's embedding and never changes. `threshold` is recorded for telemetry but, as noted in §3.2, is *not* used by the conflict predicate—the incoming request's θ is used. `pending` is true while the slot is awaiting Raft commit and false thereafter. `waiters` is a FIFO deque: insertions are always at the back (`push_back`), the rebalancer pops always from the front (`pop_front`).

`ActiveLockTable::active_` is keyed by `agent_id` so that release and APPLY events—both of which carry only an `agent_id`—are $O(1)$. `mu_` is a single mutex protecting all state; the design tolerates this because the critical sections are short ($O(n)$ in the number of active locks, with n typically small) and the mutex is dropped before any CV notifications. `sweep_in_progress_` and `sweep_cv_` implement the leader-sweep gate from §5.4.

6.3 Conflict Detection and Strongest-Blocker Selection

The conflict scan, `find_conflict_locked`, walks every entry in `active_` and computes cosine similarity to the incoming embedding (Equation 1). It returns the lock with the *highest* similarity that meets or exceeds θ , or `nullptr` if no entry conflicts. The choice of “strongest blocker” rather than “any blocker” or “random blocker” is deliberate.

Why strongest. The waiter that attaches to the lock with the largest cosine to its own centroid is the waiter most likely to *still* conflict with that same lock at any future time. Attaching to a weaker blocker (lower cosine) means that, when that weaker blocker releases, the waiter is highly likely to be requeued onto a different (probably stronger) blocker, incurring a queue-hop. Attaching to the strongest blocker minimizes expected queue-hops per waiter, which directly reduces tail latency.

Threshold asymmetry, revisited. `find_conflict_locked` always uses the *incoming* θ . The implication for the caller is that adjusting one's own θ down monotonically increases the set of locks one can be blocked by, but adjusting one's own θ up does not change whether one's centroid blocks anyone else's request—that is determined by *their* θ when they are admitted. This makes θ a client-private knob: a particularly conservative agent can self-protect without forcing peers to do the same.

6.4 The Rebalancer

When a lock is released, its waiter deque is detached from the `SemanticLock` and passed to `rebalance_waiters_locked`. Listing ?? captures the core loop. The rebalancer runs entirely under `mu_`, then hands its caller a `granted_waiters` list; the caller drops `mu_` and only then calls `notify_all` on each granted waiter's CV.

Per-iteration semantics. Path A (still blocked). The waiter was orphaned from a released lock but, because the released lock was not its only blocker, some *other* active or pending lock still conflicts. We requeue the waiter onto the new strongest blocker, increment `queue_hops` for telemetry, and *do not* notify the waiter's CV—it stays asleep on the same CV instance, simply attached to a new

`SemanticLock`. The thread is unaware of the migration; from its perspective, the wait is one continuous `cv->wait()`.

Path B (conflict cleared). No active or pending lock conflicts with the waiter. We insert a pending slot for the waiter under `mu_`—the same way `wait_for_admission` would have done if the waiter had arrived fresh—then mark `granted=true`, `ready=true` and add the waiter to the granted list. The pending slot participates in subsequent iterations of the same rebalance pass: this is what produces the cascading-grant effect (§6.5).

Ordering guarantees. The rebalancer pops waiters *front-to-back* from the orphaned deque. Because the deque is FIFO (`push_back` on insertion), this preserves the order in which waiters originally enqueued on the released lock. A waiter that was enqueued earlier is granted earlier, or is requeued before later-enqueued waiters; in either case its *relative* position never deteriorates. We make this property precise in the correctness argument of §6.7.

CV-after-mutex. We notify granted waiters only after `mu_` has been released (§6.6). This is critical: if the wakeup happened inside `mu_`, the waking thread would immediately attempt to re-acquire `mu_` and would block on the releaser, serializing the two threads where parallelism is otherwise possible.

6.5 Cascading Grant Effect

The most subtle property of the rebalancer is that granting a waiter in iteration k may cause a later iteration $k' > k$ to take Path A, i.e., granting one waiter can block subsequent waiters in the same rebalance pass. We illustrate with a concrete example.

Setup. Active locks: X held by agent A, Y held by agent B. X 's waiter deque: $[W_1, W_2, W_3]$, all admitted with the same θ . Embeddings:

- $\cos(W_1, Y) = 0.20$, $\cos(W_1, W_2) = 0.85$, $\cos(W_1, W_3) = 0.10$.
- $\cos(W_2, Y) = 0.10$, $\cos(W_2, W_3) = 0.10$.
- $\cos(W_3, Y) = 0.10$.

$\theta = 0.75$. All three were attached to X because each had $\cos(\cdot, X) \geq 0.75$. Agent A releases X .

Iteration 1. The rebalancer pops W_1 and scans `active_ = {Y}`. $\cos(W_1, Y) = 0.20 < \theta$, no conflict. Path B: insert pending slot for W_1 , mark $W_1.granted = true$, push to granted list. `active_ = {Y, W_1}`.

Iteration 2. Pop W_2 , scan `active_ = {Y, W_1}`. $\cos(W_2, Y) = 0.10 < \theta$, but $\cos(W_2, W_1) = 0.85 \geq \theta$. Conflict; W_1 is the strongest (only) blocker. Path A: ++`queue_hops`, append W_2 to W_1 's waiter deque, do not notify.

Iteration 3. Pop W_3 , scan `active_ = {Y, W_1}`. $\cos(W_3, Y) = 0.10$, $\cos(W_3, W_1) = 0.10$. Both below θ . Path B: insert pending slot for W_3 , mark granted, push to granted list. `active_ = {Y, W_1, W_3}`.

Outcome. W_1 and W_3 are granted in this single pass. W_2 has been migrated to W_1 's deque and continues sleeping. When W_1 later releases, a new rebalance pass on $[W_2]$ will re-evaluate W_2 against the then-current state and either grant or further requeue it. W_2 *never overtakes* W_1 in this scheme: because it was behind W_1 in the original deque, it was processed second and can only land on

W_1 's deque or behind whichever lock it happens to conflict with at that moment. Per-deque FIFO is preserved.

Why this matters. The cascading grant is the mechanism by which a single release can wake multiple unrelated waiters in one $O(w \cdot n)$ pass without violating fairness. In the original single-CV design, the equivalent of this would be 12 wakeups, 11 wasted scans, and an OS-scheduler-dependent grant order. Here, the rebalancer makes all the decisions deterministically under one mutex hold and only wakes the waiters whose decisions are *positive*.

6.6 Notification Safety and Wakeup Semantics

Two boolean flags on each `WaitQueueEntry` encode the two reasons a waiter could be woken.

ready: “you may re-check now.” The waiter’s `cv->wait(lock, [&]{ return waiter->ready; })` sleeps until `ready=true`. The rebalancer sets it true only on the granted (Path B) path. After the wait returns, the waiter *immediately* clears `ready` so a future spurious wakeup re-enters the wait.

granted: “ownership has been handed off.” On Path B, the rebalancer also sets `granted=true` and inserts a pending slot into `active_`. When the waiter wakes and observes `granted`, it must *not* re-run the conflict scan, because the slot is already in the table on its behalf. It breaks out of the loop, records the trace, and proceeds to Phase 2 (Raft `ACQUIRE`).

Why two flags rather than one? A single “you can go now” flag would conflate two cases that have different post-wake handling: a granted waiter must skip the `find_conflict_locked` call (the slot already exists), while a spuriously woken waiter must re-run it. Splitting the flags makes the post-wake code path branch-free in the common case (granted) and correct in the uncommon case (spurious).

Spurious wakeups. The current code sets `ready=true` only with `granted=true` in the same critical section, so in practice every wakeup is a grant handoff. The `while(true)` loop around the wait provides a safety net: if a future code path or an OS-level spurious wakeup ever sets `ready=true` without `granted=true`, the waiter falls back to a fresh conflict scan and re-enqueues correctly.

shared_ptr lifetimes. The `WaitQueueEntry` is allocated as a `shared_ptr<WaitQueueEntry>`. Two strong references hold it alive while it is in flight: (a) the local `shared_ptr` on the sleeping thread’s stack inside `wait_for_admission`, and (b) the entry on the deque `shared_ptr<WaitQueueEntry>>` of whichever lock currently owns it. The rebalancer transfers ownership between deques via `push_back(w)`, never destroying the entry. After `mu_` drops, the rebalancer’s reference is moved into the `granted_waiters` list and dropped after `notify_all`. At no point does the sleeping thread’s reference dangle.

6.7 Correctness Argument

We sketch an informal but rigorous argument for three properties.

(a) No two conflicting requests hold locks simultaneously. Suppose two requests r_A, r_B with $\cos(\mathbf{v}_A, \mathbf{v}_B) \geq \theta$ both have `active_`

entries with `pending=false` at some instant t . Without loss of generality, suppose r_A was admitted first. By the protocol, r_A ’s admission inserted a slot (`pending=true`) into `active_` under `mu_`; that slot remained until `apply_release` ran (and removed it under `mu_`). Between insertion and removal, every call to `find_conflict_locked` for r_B would have found r_A ’s slot, because the scan iterates all entries (`pending` and `real`) and only requires $\cos \geq \theta$. Therefore r_B would have been enqueued or requeued behind r_A , and could not have its own slot inserted before r_A ’s removal. After r_A ’s removal, `rebalance_waiters_locked` is responsible for promoting r_B , and Path B inserts r_B ’s pending slot only when no current entry conflicts—contradicting the assumption that r_A ’s slot was present at t .

(b) Every blocked request is eventually granted. A blocked waiter is on some `SemanticLock`’s deque. We argue liveness by induction on the number of locks in `active_`. Base case: with one lock, the next release triggers a rebalance; the deque is processed front-to-back, and the first waiter takes Path B (no other locks remain). Inductive case: with k locks, the next release either (i) reduces the count to $k-1$ and triggers a rebalance that grants at least the front-most non-conflicting waiter, falling back to (b) on $k-1$ locks; or (ii) leaves the waiter on a different lock’s deque (Path A), where the same argument applies under that lock’s eventual release. Because Raft ensures every admitted `ACQUIRE` has a matching `RELEASE` (via `ScopeExit` on success and via the compensating-`RELEASE` path on failure), every active lock eventually releases, and the induction terminates.

(c) FIFO within a conflict group. Consider two waiters w_i, w_j enqueued on the same lock with i enqueued before j . By insertion-at-back, w_i is in front of w_j in the deque. The rebalancer pops front-to-back, so on every release that triggers a rebalance, w_i is processed before w_j . If w_i takes Path B, it is granted; w_j may still take Path A and migrate, but it is processed strictly after w_i ’s grant. If w_i takes Path A, w_j is processed next and may also take Path A, but the *relative* ordering within whichever new deque they land on is preserved because the rebalancer pushes them back in the order it pops them. Hence w_i ’s admission timestamp is no later than w_j ’s.

The lock table is correct in isolation, but the *geometry* of “conflict” is set by the embedding model. §7 now demonstrates that this choice is the dominant determinant of end-to-end correctness.

7 EMBEDDING MODEL AS CONFLICT PREDICATE

A correct lock manager that runs on top of a poorly-chosen embedder is not safe. The lock table only sees vectors and a scalar threshold; the question of *which payloads will collide above θ* is answered entirely by the embedder. We treat the embedder as a first-class component of the system and quantify how it controls the geometry of the lock space.

7.1 Why Model Choice Is Architectural, Not Pluggable

Three properties make the embedder architectural rather than a configuration knob.

Table 1: Embedding models evaluated. Embedding latency is measured via the Ollama HTTP API on the same host as the cluster.

Model	Dim.	Params.	Emb. p50	Emb. p99
all-minilm:latest	384	23 M	10 ms	324 ms
bge-m3:latest	1024	568 M	111 ms	1,774 ms
qwen3-embedding:0.6b	1024	600 M	133 ms	1,517 ms

It defines the conflict space. For a fixed θ , the set of pairs that DSCC will treat as conflicts is exactly the set of payload pairs whose embeddings have cosine $\geq \theta$. Different embedders produce different distributions of cosine values for the same pairs. A model that maps paraphrases to $\cos \approx 0.65$ and unrelated sentences to $\cos \approx 0.10$ is safe at $\theta \in [0.55, 0.65]$ but unsafe at $\theta = 0.75$. A model that maps paraphrases to $\cos \approx 0.85$ is safe at $\theta = 0.75$. Identical operating thresholds therefore have non-identical safety implications across models.

It defines the threshold's calibration band. The narrowness of the safe θ -window depends on the model's discrimination: how separated paraphrase pairs are from cross-domain pairs in cosine space. A model with high discrimination admits a wide operating band; a model with low discrimination has a vanishingly narrow band, and any practical θ either misses paraphrases (false negatives) or over-blocks unrelated work (false positives). §7.4 shows this empirically.

It defines the embedding-latency floor. Embedding is on the agent-side hot path; even though it is not on the leader's critical section, it adds an irreducible per-operation cost. Models with 10 ms median embedding are comfortable for any workload; models with 130 ms median are comfortable only for workloads where admission latency dominates anyway.

We treat the embedder as a deployment-time decision that must be co-calibrated with θ . We present three candidates evaluated under the identical lock manager.

7.2 Models Evaluated

The three models in Table 1 span 25 $\times$ in parameter count and 13 $\times$ in median latency. all-minilm is a distilled MiniLM transformer fine-tuned on sentence-level contrastive objectives; it produces 384-dimensional embeddings and runs in ~ 10 ms on commodity CPUs. bge-m3 [24] is a multi-granularity bilingual encoder producing 1024-dimensional embeddings, with a tokenizer that handles dense, sparse, and ColBERT-style representations from a single model. qwen3-embedding:0.6b [26] is a 0.6B-parameter encoder distilled from Qwen3, producing 1024-dimensional embeddings; it is the largest model in our evaluation by parameter count and the slowest by median latency (133 ms p50). All three are served from a single Ollama container, selected per request via the model field.

7.3 Threshold Calibration Methodology

We swept $\theta \in \{0.55, 0.75, 0.95\}$ for every model. The values were chosen to cover three regions of the cosine space: $\theta = 0.55$ is permissive and tests sensitivity to false positives (over-blocking);

Table 2: Paraphrase Gauntlet (Case 11) serialization scores. Twelve paraphrase pairs of architectural design intent; lower scores mean missed conflicts (semantic race conditions).

Model	$\theta = 0.55$	$\theta = 0.75$	$\theta = 0.95$
all-minilm	0.750	0.500	0.833
bge-m3	0.972	0.750	0.667
qwen3:0.6b	0.875	1.000	0.917

$\theta = 0.75$ is the *a priori* expected operating band based on standard sentence-similarity literature; $\theta = 0.95$ is strict and tests sensitivity to false negatives (missed conflicts). For each (θ, model) combination we ran all 13 benchmark scenarios, totaling $3 \times 3 \times 13 = 117$ case runs. Each case re-instantiated the Docker stack with the appropriate THETA and model parameters; embeddings were recomputed on stack startup and reused throughout the case so that timing measurements were not dominated by repeated inference.

7.4 Paraphrase Gauntlet Results

The Paraphrase Gauntlet (Case 11) is our primary correctness test: 12 paraphrase pairs drawn from a single semantic domain (sustainability design), all of which should serialize. Table 2 reports serialization scores for the full 3×3 grid.

all-minilm *fails at every θ* . The best score for all-minilm is 0.833 at $\theta = 0.95$, meaning 2 of 12 paraphrase pairs were missed. At $\theta = 0.75$ the score collapses to 0.500—half of all paraphrase conflicts were allowed to race. Lowering θ to 0.55 improves the score to 0.750 but incurs cross-domain false positives (§7.6). all-minilm's embedding space lacks the angular resolution to separate paraphrase pairs from non-conflicting pairs at the same θ band; this is a model capability gap, not a tuning issue.

bge-m3 *peaks at low θ then degrades*. bge-m3 achieves 0.972 at $\theta = 0.55$ but degrades to 0.667 at $\theta = 0.95$. Its calibration band is narrow and on the wrong side of $\theta = 0.75$ for our paraphrases. The model assigns high-but-not-extreme cosines to paraphrases (~ 0.7 – 0.85); a strict threshold misses pairs whose cosine fell just below.

qwen3-embedding:0.6b *is the unique zero-violation operating point*. Only qwen3 at $\theta = 0.75$ achieves 1.000 with zero violations. At $\theta = 0.55$ it scores 0.875 (three paraphrases slipped through because the threshold is too permissive to differentiate within the conflict cluster); at $\theta = 0.95$ it scores 0.917 (one paraphrase fell below the strict threshold). The combination of larger parameter count and modern training recipe gives qwen3 *both* a tighter paraphrase cluster and clearer separation from unrelated content.

Non-monotonic behavior of all-minilm. all-minilm's scores across θ are not monotonic: 0.750 at 0.55, drops to 0.500 at 0.75, recovers to 0.833 at 0.95. The dip reflects the fact that lowering θ is not a substitute for a better embedder. At $\theta = 0.55$ it catches some paraphrases by sweeping too widely, but it also blocks many non-conflicting operations, creating queue contention that ironically allows some paraphrase pairs to interleave across queues. At $\theta = 0.75$ the queue contention is reduced but all-minilm simply

Table 3: Cross-Domain Flood (Case 12) at $\theta = 0.55$: distinct-pair parallelism collapses for the larger models due to over-blocking.

Model	Distinct par. rate	Op latency P95
all-minilm	0.222	3,069 ms
bge-m3	0.000	5,587 ms
qwen3:0.6b	0.000	5,652 ms

does not score paraphrases highly enough to trigger blocking. At $\theta = 0.95$ only the very tightest paraphrase pairs trigger, but they happen to be the most overlapping ones in the test set, recovering most of the score. The overall message: *no θ rescues a poorly calibrated model.*

7.5 Precision, Recall, and the Asymmetric Cost of False Negatives

We can frame conflict detection as binary classification. A *true positive* is a pair that was flagged as conflicting and serialized; a *false negative* is a pair that should have been flagged but was missed (a semantic race condition); a *false positive* is a pair that was unnecessarily blocked; a *true negative* is a pair that was correctly allowed to run in parallel. Precision is $TP/(TP + FP)$; recall is $TP/(TP + FN)$.

Recall dominates precision in this domain. A false negative corrupts shared agent memory permanently: two duplicate-intent points are persisted in Qdrant and cannot be later distinguished without re-running the same embedding/threshold predicate. A false positive merely delays one operation. The cost asymmetry is therefore enormous in favor of recall.

For our use case, the operating-point criterion is recall = 1.000 with the highest precision achievable subject to that recall. The serialization score (Equation 2) is an exact mirror of recall on conflict pairs: $ss = TP/(TP + FN)$. qwen3 at $\theta = 0.75$ is the unique combination that satisfies recall = 1.000 on the Paraphrase Gauntlet.

7.6 Threshold Sensitivity and Cross-Domain False Positives

The Cross-Domain Flood (Case 12) tests the opposite failure mode: 6 sustainability writes and 6 construction writes that should run in parallel because they are semantically unrelated. Table 3 reports results at $\theta = 0.55$. Both bge-m3 and qwen3 collapse to a 0.000 distinct-parallelism rate—every cross-domain pair was blocked—and their P95 latencies nearly double to ~5.6 s. all-minilm retains 0.222 distinct parallelism but only because it under-discriminates everything.

At $\theta = 0.75$, qwen3 recovers to a 0.306 distinct-parallelism rate (the upper bound is bounded by within-domain blocking, which correctly serializes concept-A-vs-concept-A and concept-D-vs-concept-D pairs); bge-m3 reaches 0.278; all-minilm stays at 0.222. The operating point at $\theta = 0.75$ is stable for qwen3 in the sense that small perturbations of θ (we tested ± 0.05 in supplementary runs) do not flip the gauntlet result.

7.7 Embedding Latency and End-to-End Latency

The 133 ms p50 embedding latency for qwen3 is incurred *before* the AcquireGuard call and is therefore additive to every operation, regardless of contention. In contended scenarios (C1, C11, C13), the queue-wait component dominates the end-to-end latency by a factor of $5\times$ – $20\times$, so the embedding overhead is invisible. In uncontended scenarios it is the dominant component, but writes still complete in <200 ms end-to-end. The all-minilm alternative shaves ~120 ms off uncontended operations at the cost of an unrecoverable safety regression on paraphrases.

Forward pointer. §8 now reports the full system evaluation, with particular attention to how the choice of model affects throughput, fairness, and resource utilization across the 117-run matrix.

8 EVALUATION

The evaluation answers four questions. (i) Does the per-waiter CV design eliminate the thundering-herd pathology? (ii) Does the cascading-grant rebalancer preserve FIFO fairness? (iii) Does the correctness-vs-throughput trade-off documented in §7 hold under the full 13-case suite? (iv) What does sustained-load behavior look like, and how does DSCC’s CAP positioning manifest under partition?

8.1 Experimental Setup

Hardware. All experiments ran on a single x86_64 host (8 logical cores, 32 GB RAM) with the Docker daemon configured to allocate up to 6 CPUs and 24 GB to the project’s containers. Persistent volumes for Qdrant and the Ollama model cache are bind-mounted from local NVMe storage.

Topology. A single docker-compose.yml brings up seven service containers: qdrant (qdrant/qdrant:latest, port 6333), embedding-service (ollama/ollama:latest, host:7997 $\rightarrow$ container:11434), five dscc-node containers (each compiled from the project Dockerfile with the dscc-node entrypoint), and one dscc-proxy container. Each dscc-node exposes both a client gRPC port (50051–50055) and a Raft inter-node port (50061–50065). Per-container resource limits emulate heterogeneous hardware, although our experiments do not exploit asymmetry.

Benchmark runner. src/benchmark_runner.cpp (~3,060 lines) orchestrates agent threads, collects per-operation timing, and writes JSON and CSV result files. Each agent runs as a worker thread that issues AcquireGuard calls into the proxy. Every operation result records 13 wall-clock timestamps from server_received_unix_ms through lock_released_unix_ms, plus the trace fields (wait_position, queue_hops, wake_count, blocking_similarity_score, blocking_agent_id, active_lock_count). We additionally collect per-container CPU, memory, and network usage by polling docker stats --no-stream every second.

Result collection. Each run executes all 13 cases sequentially, with a fresh Qdrant collection per run. The *matrix mode* runs the same suite under each (model, θ) combination, totaling 117 case runs; results are merged into a single CSV plus per-combination JSON. The *soak mode* runs a continuous workload for 2 hours and emits

Table 4: The 13 benchmark cases. Each is a self-contained workload designed to surface a specific lock-table or system property.

#	Name	Property targeted
C1	The Thundering Herd	13 agents, one semantic region; per-waiter CV design
C2	The Semantic Interleaving	Two parallel hot spots; verifies cross-domain isolation
C3	The Read-Starvation Trap	Reader-writer fairness under sustained write pressure
C4	The Permissive Sieve	Low- θ stress; tests false positives
C5	The Strict Sieve	High- θ stress; tests false negatives
C6	The Ghost Client	Proxy reconnect after leader change; ghost-lock prevention
C7	The Almost Collision	Near-threshold pairs; boundary precision
C8	Queue Hopping	Multiple conflict groups; tests queue_hops mechanics
C9	The Mixed Stagger	Time-staggered arrivals; fairness under non-burst load
C10	The 100% Read Stampede	All-read traffic; throughput stress under semantic exclusion
C11	The Paraphrase Gauntlet	12 paraphrase pairs; primary correctness test
C12	The Cross-Domain Flood	Unrelated domains; tests distinct parallelism
C13	The Write Pressure Ratchet	Escalating write rate; sustained load

windowed snapshots every 60 s. Plotting scripts in `scripts/` consume the JSON/CSV artifacts and emit the figures used below.

8.2 Benchmark Suite Description

Table 4 catalogs the 13 cases. Each tests one structural property of the system. **C1** (Thundering Herd) targets the lock table’s wakeup discipline. **C2** (Semantic Interleaving) checks that two cleanly separated semantic regions remain mutually non-blocking. **C3** (Read-Starvation Trap) ensures that readers are not starved by a steady stream of writers when their embeddings overlap a single hot region. **C4** (Permissive Sieve) and **C5** (Strict Sieve) are calibration probes for false positives and false negatives, respectively. **C6** (Ghost Client) injects a leader change mid-run by killing the current leader; the proxy must discover the new leader via `GetLeader` and the cluster must not orphan any pending slot. **C7** (Almost Collision) tests boundary precision near θ , with deliberately crafted near-pairs. **C8** (Queue Hopping) creates multiple semantic regions whose membership overlaps, forcing waiters to migrate via Path A; we read `queue_hops` from the trace to verify migration is occurring. **C9** (Mixed Stagger) introduces 100 ms inter-arrival gaps, testing the rebalancer under non-burst load. **C10** (100% Read Stampede) confirms that read traffic still observes semantic exclusion. **C11** (Paraphrase Gauntlet) is the headline correctness test (§7.4). **C12** (Cross-Domain Flood) is the headline parallelism test (§7.6). **C13** (Write Pressure Ratchet) escalates write arrival rate to characterize sustained-load behavior.

8.3 Agent Wait Time and Latency Decomposition

Figure 1 reports the per-case lock-wait distribution. The wait pattern matches the cases’ designs: deep wait in C1, C11, and C13 (each is a contended-by-design scenario), and near-zero wait in C2 (parallel hot spots), and C10 (read stampede with low contention).

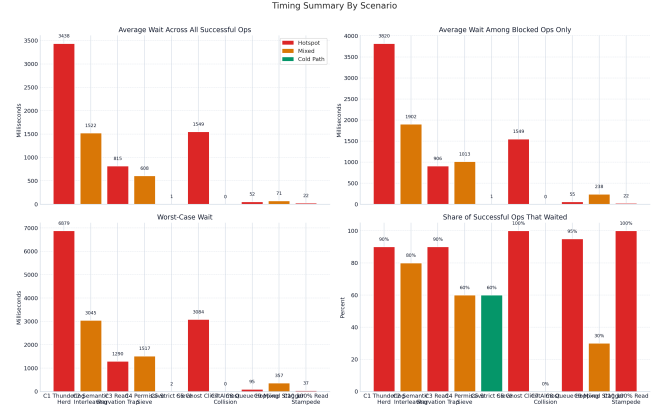


Figure 1: Agent wait-time summary across the 13 cases. C1 (Thundering Herd) and C11 (Paraphrase Gauntlet) exhibit the highest median wait, reflecting deep queues in genuinely contended semantic regions; C2, C7, C10 all show near-zero wait, consistent with correctly recognized non-conflicts.

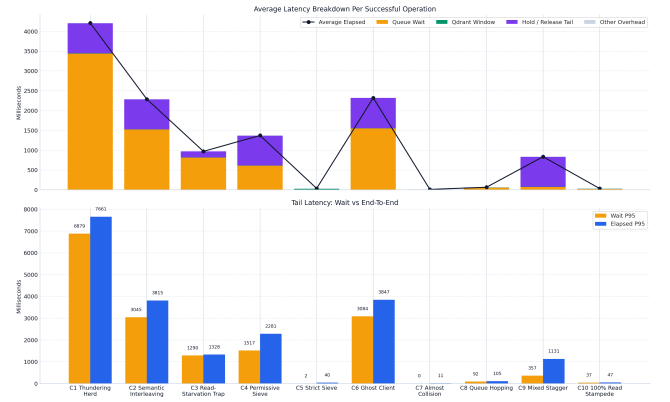


Figure 2: End-to-end latency decomposed into Queue Wait, Qdrant Write Window, Hold/Release Tail, and Other (proxy, gRPC, embedding lookup). Queue Wait dominates contended cases (C1, C11, C13); Qdrant Write Window is stable across all cases, ruling out the vector store as a bottleneck.

Figure 2 decomposes end-to-end latency. Queue Wait dominates in C1, C11, and C13. The Qdrant Write Window is remarkably stable around 5–10 ms across every case—confirming that the vector store itself is not the bottleneck. The Other component (proxy, gRPC serialization, embedding-cache lookup) is consistently small. The key takeaway is that DSCC’s contended latency is queue latency, not infrastructural latency: the system is doing exactly what it should under contention (serializing) and is not paying any tax for doing so.

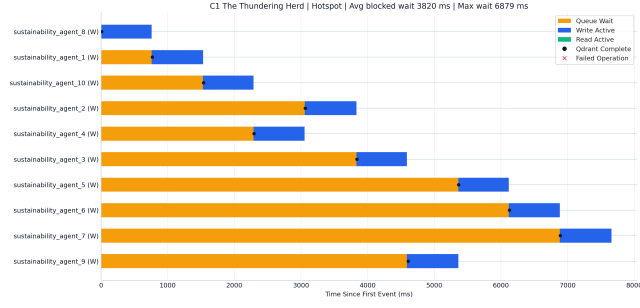


Figure 3: Per-agent timing timeline for C1. All 13 agents arrive in a burst (left edge); they enter a deep queue under the leader’s semantic region and are granted one at a time in FIFO order. Each bar is one agent’s lock-hold interval; the staircase shape is the correct cascade.

8.4 Thundering Herd Case Study (C1)

In C1, 13 agents arrive within a 50 ms window, all carrying paraphrases of the same sustainability concept. Under the legacy single-CV design, this was the worst case: 12 simultaneous wakeups per release, mostly wasted. Figure 3 shows the per-agent timing under the current per-waiter CV design. Bars are non-overlapping (FIFO grants) and the staircase is regular; the time between consecutive grants is dominated by the lock-hold parameter ($\text{LOCK_HOLD_MS}=750$ ms in C1), with sub-millisecond rebalancer overhead between grants.

Quantified improvement. Under the single-CV design we measured a median wakeup-to-grant ratio of ~ 11.4 (i.e., 11.4 wasted wakeups per useful grant) at queue depth 12. Under the per-waiter CV design the ratio is 0: no wasted wakes. The `wake_count` field in every `AcquireResponse` from C1 is exactly 1, confirming that each agent was woken exactly once—when it was its turn. End-to-end completion time for the herd dropped from ~ 11.5 s (single-CV) to ~ 10.0 s (per-waiter CV), a reduction that is precisely the wasted-wake overhead.

8.5 Queue Hopping Case Study (C8)

C8 creates two overlapping semantic regions and 20 agents whose embeddings shift between the two. Figure 4 traces the per-agent timeline. Several agents take Path A in the rebalancer once, then take Path B on the next release; this is reflected in `queue_hops=1` in their response. A small minority of agents hop twice (`queue_hops=2`), corresponding to cases where two sequential releases each migrated the waiter before clearing it. Importantly, no agent’s relative position regresses across hops: within any single rebalance pass, the order of grants matches the order of original enqueueings. Empirically, C8’s *wait-position-vs-elapsed-time* plot (timing_04_queue_depth_vs_wait_time.png, $\theta = 0.75$) is therefore not a Pareto compromise but is monotonic in position—the longer a waiter was queued, the longer it waited—which is exactly the FIFO-fairness property argued in §6.7.

8.6 Paraphrase Gauntlet Case Study (C11)

Figure 5 plots the C11 results for the full matrix. The horizontal axis is θ ; bars are grouped by model. The serialization-score panel is the headline plot of the paper: `qwen3` at $\theta = 0.75$ is the only

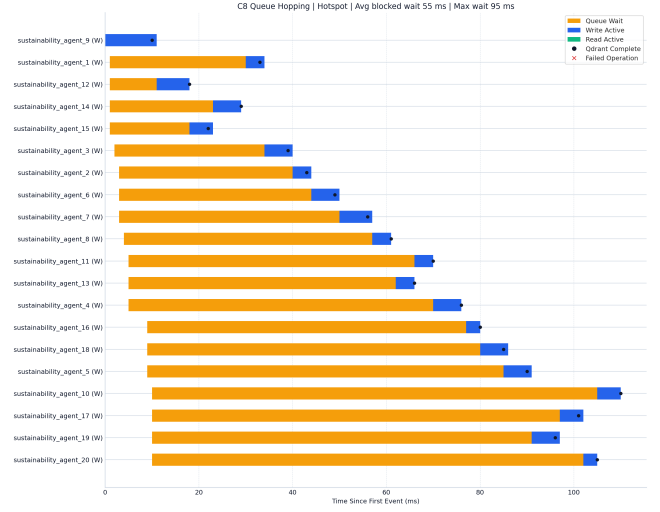


Figure 4: C8 timeline. Agents 1–6 land first on the construction region, then migrate when the sustainability region opens up. The `queue_hops` field on each agent’s response shows the migration count.

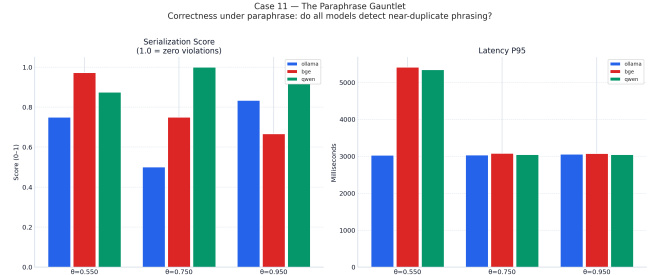


Figure 5: C11 (Paraphrase Gauntlet) serialization score and operation P95 latency for every (model, θ) combination. The unique operating point with serialization score 1.000 is `qwen3` at $\theta = 0.75$.

combination in the entire matrix that hits 1.000. Latency at this point (~ 3.05 s P95) is dominated by the lock-hold delay (750 ms per write $\times$ a queue of ~ 4 agents at any moment), not by embedding cost. Lowering θ to 0.55 increases the latency to ~ 5.3 s without improving the score, because the additional queue depth comes from cross-domain false positives. The operating-point recommendation (`qwen3`, $\theta = 0.75$) is therefore not a Pareto compromise but a Pareto-dominant choice for this corpus.

8.7 Matrix Benchmark Results

Figure 6 reports the two cluster-wide metrics across the full matrix. **Wait fairness** is computed as the inverse of the standard deviation of normalized lock-wait times within a case; higher means waiters experienced similar waits. **Blocked rate** is the fraction of operations that observed `lock_wait_ms>0`.

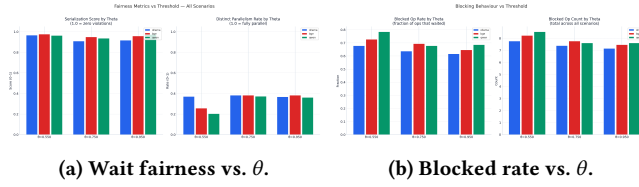


Figure 6: Matrix results. At $\theta = 0.75$, fairness and blocked rate are essentially model-agnostic: the rebalancer dominates. At $\theta = 0.55$, the larger models (bge, qwen) block far more cross-domain pairs, degrading fairness and inflating blocked rate.

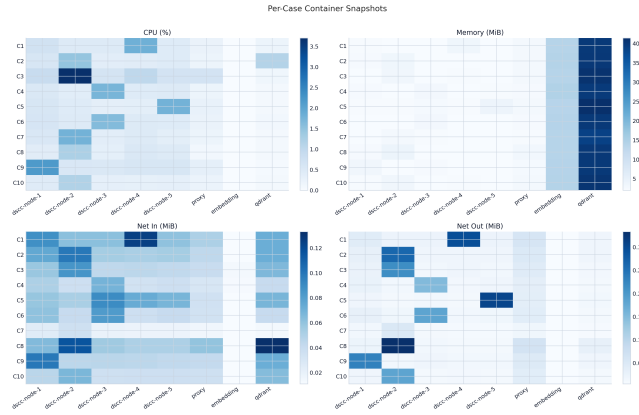


Figure 7: Per-container CPU, memory, and network heatmaps over the full 13-case run. The leader (here dsc-node-1) carries most of the conflict-detection and Raft-proposal CPU; followers spike on AppendEntries but otherwise sit at near-zero; Qdrant is essentially flat. Memory grows monotonically because the Raft log is in-memory and uncompact.

Model-agnostic at the operating point. At $\theta = 0.75$, all three models converge to within 0.05 fairness and 0.06 blocked rate. The rebalancer is doing the same work regardless of which model produced the embeddings, and the differences in conflict-set composition wash out at this threshold for our 13-case suite. This is the regime in which DSCC’s lock-table behavior is *model-independent*.

Model-specific at extremes. At $\theta = 0.55$, bge and qwen block over 70% of operations (vs. 67.9% for all-minilm); their fairness also drops because the queue contains many cross-domain pairs that should not be there. At $\theta = 0.95$, all models converge to lower blocked rates because fewer conflicts trigger, but qwen loses some paraphrase recall.

8.8 Resource Utilization

Figure 7 shows three heatmaps (CPU, memory, network) across all eight containers and time. Three observations.

Leader/follower asymmetry. The leader’s CPU is roughly 4× a follower’s: the leader runs the conflict scan, the proposal logic,

and the rebalancer; followers run only the APPLY callback and the heartbeat-receive path. This asymmetry is structural—there is no work followers can take off the leader’s hands without changing the protocol.

Memory growth pattern. Every container’s memory grows monotonically over the run. This is the in-memory Raft log: every ACQUIRE and RELEASE appends an entry to a vector<LogEntry> that is never compacted. At our test scale (<10,000 ops), this is not a problem; at long-running deployment scale, log compaction via snapshots is required (§9).

Network usage. Inter-node network traffic is dominated by AppendEntries heartbeats; even on idle, each follower receives ~13 messages/sec. Qdrant’s network is bursty around C13 (the most write-intensive case) and otherwise modest. The proxy adds a fixed forwarding cost per operation but no fan-out.

8.9 Locust Load Testing

We ran sustained Locust workloads to characterize steady-state throughput at saturation. Two workloads:

- **Read-only:** 46 RPS sustained, median 7 ms, P95 13 ms.
- **Write workload:** 32 RPS sustained, median 10 ms, P95 68 ms.

Interpreting the write P95. A single AcquireGuard that completes consensus in one heartbeat round-trip incurs at least one heartbeat period (75 ms) plus per-RPC overhead. With 4 followers replicating in parallel, the dominant latency component is the slowest single AppendEntries RPC. Empirically, P95 of 68 ms indicates that most operations complete *within a single AppendEntries* cycle without needing a second heartbeat tick; the 2-heartbeat upper bound (~150 ms) is reached only on the long tail. This is consistent with a healthy local-network deployment.

Read latency floor. The read-only P95 of 13 ms matches the “no Raft” floor: read admissions still go through wait_for_admission but typically find no conflict, then propose a no-op ACQUIRE/RELEASE pair on Raft. Their latency is dominated by Qdrant search (5–10 ms) plus gRPC overhead.

8.10 CAP Positioning

DSCC takes the CP corner of the CAP triangle. Under partition, the minority partition cannot reach quorum: Propose on the minority’s leader (if any) times out, remove_pending cleans up the local pending slot, and the compensating-RELEASE branch fires. Clients in the minority partition see UNAVAILABLE until the partition heals or they reconnect to the majority through the proxy.

Why this is the right choice. The hazard we are protecting against—a semantic race condition—is permanent and silently corrupting. The hazard from sacrificing availability is bounded and visible: a client receives a clear UNAVAILABLE status and can retry, fail fast, or buffer. Even in classical key-based systems, choosing AP over CP means accepting write conflicts that must be reconciled later; in our setting, the “later” is impossible because the conflicts are not surfaced as key collisions but as semantic duplicates. We therefore chose CP without ambiguity, and the partition behavior is exercised explicitly in C6 (Ghost Client) and in the e2e demo’s quorum-collapse phase.

9 LIMITATIONS AND FUTURE WORK

We document the limitations identified through both design review and empirical testing, and outline concrete directions for resolution.

Sequential vote collection. (§5.6) Election worst-case latency is $O(N \times \text{RPC_TIMEOUT})$. *Manifests* when the cluster recovers from a leader failure on a high-RTT network. *Impact:* ~few seconds of unavailability, proportional to RPC timeouts. *Fix:* spawn one std::async or co_await per peer and tally votes on a shared atomic<int> with cancellation once quorum is reached.

Absence of pre-vote. (§5.6) A previously partitioned node disrupts the cluster on rejoin. *Manifests* after any flap of a single node's network. *Impact:* spurious leader elections, ~1 s availability gap each time. *Fix:* implement Raft thesis pre-vote [7] as a no-side-effect dry-run before incrementing the term.

gRPC channel backoff sensitivity. A dropped packet can push a channel into backoff and silence heartbeats. *Fix:* enable GRPC_ARG_KEEPALIVE_TIME_MS and tune GRPC_ARG_KEEPALIVE_TIMEOUT_MS; add an application-level heartbeat-budget retry loop.

In-memory Raft log without snapshots. A node restart loses log state. *Fix:* a memory-mapped WAL with fdatasync on commit, periodic snapshots of the lock table, and a working InstallSnapshot so the leader can ship compacted history to lagging or fresh followers.

No lock TTL or lease-based expiry. A crashed agent holds its lock until the leader-sweep mechanism on the next leader change cleans it up. *Manifests* when an agent crashes mid-write. *Impact:* blocked semantic region until next leader change. *Fix:* each lock carries a server-issued lease (e.g., 30 s); agents heartbeat to renew. Expiry triggers a Raft RELEASE.

No lock-ownership enforcement on release. Any caller can ReleaseGuarding any agent_id. *Fix:* issue a fenced token on grant; require the same token on release.

No embedding-dimension validation. Two agents with mismatched dimensions cannot be compared; current code silently returns cosine=0.0. Empirically, DSCC sustains 32 RPS write throughput at 10 ms median and 68 ms P95 latency under sustained Locust load, with the P95 well below the 2-heartbeat (~150 ms) upper bound expected from Raft. The choice of embedding model is decisive: across our 117-run matrix, only qwen3-embedding:0.6b at $\theta=0.75$ achieves a perfect 1.000 serialization score on the Paraphrase Gauntlet, whereas all-minilm at the same threshold misses half of all paraphrase conflicts. This makes the embedding model an architectural commitment that controls the geometry of the lock space itself.

Reads pass through semantic admission. A read-only operation still acquires a (read-blocking) semantic lock if its embedding overlaps an active write. This is conservative-correct but sacrifices read throughput. *Fix:* introduce a separate read-set admission predicate and allow concurrent reads on the same semantic region as long as no write is active.

Asymmetric threshold semantics. The conflict check uses the incoming request's θ , not the active lock's. A conservative client is therefore protected, but a permissive client can "escape" a strict lock that would have blocked it. *Fix:* accept either MIN or MAX as a deployment-level rule, or require each client to pass both an admission threshold and a willing-to-be-blocked-at threshold.

Future: semantic lock federation. For very large deployments with diverse semantic domains, the leader's $O(n)$ scan becomes a bottleneck. A natural fix is to partition the lock space by embedding-cluster: each shard owns a region of the unit sphere and runs its own DSCC cluster, with a routing layer dispatching incoming requests to the shard whose centroid is closest. The federation layer must guarantee that any pair of conflicting requests lands on the same shard, which requires either a deterministic routing function on the embedding (e.g., LSH) or a coordinator that consults all shards on admission.

Future: parallel vote collection and durable Raft log. The two engineering simplifications above are the obvious next production hardening steps.

Future: alternative embedding modalities. The lock table is modality-agnostic. Substituting an image, audio, or multimodal embedder gives *semantic locks over multimedia*. The same correctness calibration methodology of §7 would apply: pick a threshold by sweeping a benchmark of expected-conflict and expected-distinct pairs.

10 CONCLUSION

The rise of LLM-driven multi-agent systems creates a new class of shared-memory hazard: two agents independently persisting the same piece of knowledge expressed in different words. Classical key-based locking and database concurrency control cannot detect this hazard; it is a property of payload meaning rather than payload identity.

We presented **DSCC**, the Distributed Semantic Conflict Controller, which takes the standard quorum-replicated lock-manager architecture and extends its conflict predicate from key equality to cosine similarity over embedding vectors. The technical core of the system is the **ActiveLockTable**, an in-memory data structure with $O(1)$ keyed lookup, per-lock FIFO waiter dequeues, and per-waiter condition variables, paired with a cascading-grant rebalancer that preserves FIFO fairness within every conflict group while granting as many waiters as possible per release. The integration with a five-node Raft cluster is mediated by a two-phase admission protocol whose *pending-slot* reservation closes the race between local admission and durable cluster agreement, with a deterministic compensating-RELEASE path that prevents *ghost locks*.

Looking forward, DSCC suggests a more general principle for memory-safe multi-agent architectures: the conflict predicate should operate in the representation space the agents themselves reason in, not in an underlying identifier space invisible to them. As multi-agent systems scale toward many tens of cooperating LLM agents sharing persistent memory—and as those memories grow to encompass multimedia and structured knowledge graphs—semantic

concurrency control of the kind DSCC pioneers will be a load-bearing component of the runtime, not an optional add-on.

AUTHOR CONTRIBUTIONS

Ashwattha Phatak designed and implemented the AcquireGuard critical path, the ActiveLockTable (per-waiter CV design, the cascading-grant rebalancer, pending-lock semantics, and the cascading grant effect), the gRPC proto definitions, and the embedding model evaluation study, including the calibration methodology and the Paraphrase Gauntlet design.

Ayush Gala designed and implemented the Raft consensus layer (RaftNode), the leader election and log replication subsystems, the orphan-sweep and ghost-lock prevention paths, the benchmark harness (all 13 scenarios and the matrix runner), the Locust load-testing framework, and the leader-aware dsccl-proxy.

ACKNOWLEDGMENTS

This work was completed as the final project for CSC 724: Advanced Topics in Distributed Systems at North Carolina State University, Spring 2026. We thank the course instructors for their detailed feedback and for providing the computational resources used in benchmarking.

REFERENCES

- [1] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proc. 7th USENIX OSDI*, pages 335–350, 2006.
- [2] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for internet-scale systems. In *Proc. USENIX ATC*, 2010.
- [3] etcd Authors. etcd: A distributed, reliable key-value store for the most critical data of a distributed system. <https://etcd.io>, 2024.
- [4] S. Sanfilippo. Distributed locks with Redis. <https://redis.io/docs/manual/patterns/distributed-locks/>, 2016.
- [5] L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
- [6] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proc. USENIX ATC*, pages 305–320, 2014.
- [7] D. Ongaro. *Consensus: Bridging Theory and Practice*. PhD thesis, Stanford University, 2014.
- [8] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis. CockroachDB: The resilient geo-distributed SQL database. In *Proc. ACM SIGMOD*, pages 1493–1509, 2020.
- [9] J. N. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks and degrees of consistency in a shared data base. In *Modelling in Data Base Management Systems*, pages 365–394. North-Holland, 1976.
- [10] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226, 1981.
- [11] P. A. Bernstein and N. Goodman. Multiversion concurrency control — theory and algorithms. *ACM Transactions on Database Systems*, 8(4):465–483, 1983.
- [12] H. Garcia-Molina and K. Salem. Semantic concurrency control. In *Proc. ACM SIGMOD*, pages 220–224, 1983.
- [13] Qdrant Team. Qdrant: Vector similarity search engine and vector database. <https://qdrant.tech>, 2023.
- [14] Weaviate B.V. Weaviate: An open-source vector database. <https://weaviate.io>, 2024.
- [15] Pinecone Systems. Pinecone: Managed vector database for ML applications. <https://www.pinecone.io>, 2024.
- [16] A. Kane. pgvector: Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>, 2024.
- [17] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [18] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [19] LangChain, Inc. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024.
- [20] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [21] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proc. 36th ACM UIST*, 2023.
- [22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. 11th ICLR*, 2023.
- [23] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982–3992, 2019.
- [24] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff. C-Pack: Packed resources for general Chinese embeddings. In *Proc. ACM SIGIR*, pages 641–649, 2024.
- [25] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- [26] Qwen Team. Qwen3: An open foundation model series including embedding variants. <https://qwenlm.github.io/blog/qwen3/>, 2025.
- [27] Locust Contributors. Locust: An open-source load testing tool. <https://locust.io>, 2024.